

HSNMatch — Step-by-Step Build Plan

Derived from PRD v1.0 | 14-day roadmap

Phase 1: Data Foundation (Days 1–2)

Exit criteria: Clean HSN master loaded into DB.

Day 1

1. Set up project skeleton: Django + DRF backend, `git init`, base folder structure (mirror CartVerse's layout since it's familiar).
2. Set up local Postgres instance (not Docker yet — local-first per Risk mitigation).
3. Source the HSN/SAC master list from the official GST portal / CBIC rate schedule.
 - Record the exact source URL and download date now — you need this for the README credibility requirement (Section 9).
4. Do a first-pass manual inspection of the raw file: check encoding, delimiter consistency, duplicate codes, missing rates, inconsistent code-length formatting (4/6/8-digit HSN codes are common issues).
5. Read the "Embeddings" concept guide (OpenAI or Anthropic/Voyage docs) — 1 hour. Make one standalone test API call (script or Postman) and print the vector length just to make it concrete.

Day 2

6. Write a cleaning script (pandas or plain Python) that normalizes the HSN master: consistent code formatting, stripped whitespace, deduplicated rows, standardized GST rate fields.
7. Design the DB schema:
 - `HSNCode`: code, description, gst_rate, chapter/section metadata, embedding (vector), source_updated_at
 - `ClassificationQuery`: input_text, created_at, full candidate set (JSON), top-1 result reference
 - `Correction`: query_id (FK), submitted_code, created_at
8. Write the Django model + migration for `HSNCode` (no vector field yet — that's Day 3).
9. Load the cleaned master list into Postgres via a management command (`load_hsn_master`).
10. Budget slack here — Risk #1 in the PRD flags this step as the one most likely to run long. Don't rush; if Day 2 overflows into Day 3 morning, that's fine.

Checkpoint: `SELECT count(*) FROM hsn_code;` returns a sane number with no obvious garbage rows.

Phase 2: Embedding Index (Days 3–4)

Exit criteria: Vector search returns sensible results by manual spot-check.

Day 3

1. Install pgvector: `CREATE EXTENSION vector;`
2. Add `pgvector.django` (or raw SQL) `VectorField` to `HSNCode`, migrate.
3. Write an embedding script: for each HSN row, call the embedding API on `description` (and any chapter/heading text if useful) and store the resulting vector.
4. Run it against the full master list. Note actual cost — should be trivially cheap per the NFR budget.

Day 4

5. Write the nearest-neighbor query: `ORDER BY embedding <-> %s LIMIT 5` wrapped in a small `get_candidates(query_text)` function that embeds the input and returns top-5 rows.
6. Skip index tuning entirely (HNSW/IVFFlat) — not needed at this row count, per the learning plan.
7. Manually spot-check with 10–15 varied descriptions (including the "waterproof canvas backpack with laptop compartment" example from the PRD problem statement). Confirm results are semantically reasonable, not just keyword-adjacent.
8. If results are weak, iterate on what text gets embedded (e.g., include category/chapter context) before moving on — this is the core retrieval quality gate for the whole project.

Checkpoint: Given 10 hand-picked test descriptions, the top-5 vector results plausibly contain the correct code for most of them.

Phase 3: Classification Endpoint (Days 5–6)

Exit criteria: `/api/classify/` returns ranked, explained results.

Day 5

1. Read Anthropic's docs on structured output / reliable JSON from the API.
2. Write and test a re-rank prompt against Claude (Haiku, per the latency budget) that:

- Takes the input description + top-5 vector candidates
 - Returns top-3 ranked, each with a confidence label (high/medium/low) and one-sentence reasoning
 - Explicitly instructs no markdown fences, no preamble — JSON only
3. Practice with 2–3 test prompts locally, confirm `json.loads()` succeeds reliably. Add try/except with one retry before it's wired into the real endpoint.

Day 6

4. Build `POST /api/classify/`:
 - Embed input → pgvector top-5 → Claude re-rank/explain → return top-3
 - Persist the full candidate set (not just the winner) to `ClassificationQuery` (FR-5)
5. Implement fallback behavior from Section 8:
 - Embedding API failure → clear service-unavailable error, no silent keyword fallback
 - LLM re-rank failure (embeddings succeeded) → return raw top-3 by similarity, labeled "unranked — explanation unavailable"
6. Add the persistent disclaimer field/flag to the response payload (FR-7): suggestion only, not a filing-ready determination.
7. Test latency end-to-end; confirm it's comfortably under ~4 seconds.
8. Build `GET /api/hsn/{code}/` direct lookup endpoint (FR-6) — simple, no LLM involved.

Checkpoint: A single curl/Postman call to `/api/classify/` returns a well-formed top-3 response with reasoning, and you can force each fallback path and see the documented behavior.

Phase 4: Bulk + Feedback (Days 7–8)

Exit criteria: CSV bulk works; corrections stored.

Day 7

1. Build `POST /api/classify/bulk/`: accepts CSV upload, requires a `description` column.
2. Process row-by-row, reusing the single-item classify path (top-1 + confidence only, per FR-3 — no need to re-rank/explain per row at full depth if that risks the 60-second budget; benchmark this).
3. Implement per-row failure handling: skip and flag the row in the output CSV rather than dropping it silently (Section 8).

4. Benchmark against a real 50-row test CSV — confirm end-to-end time is under 60 seconds. If not, look at batching the embedding calls or parallelizing.

Day 8

5. Build `POST /api/corrections/`: accepts a `query_id` + submitted correct code, stores it (FR-4).
6. Implement the "simple heuristic boost" mentioned in Scope (not fine-tuning — V2 territory): e.g., if a correction exists for a very similar past query, slightly boost that code's rank on future similar queries. Keep this deliberately lightweight; document it as a heuristic, not learning.
7. Write a few tests covering: bulk happy path, bulk with a malformed row, correction submission, correction influencing a subsequent similar query.

Checkpoint: A 50-row CSV round-trips in under 60 seconds with at least one intentionally bad row, and submitting a correction is persisted and retrievable.

Phase 5: Frontend (Days 9–10)

Exit criteria: Full user flow usable end-to-end locally.

Day 9

1. Scaffold React + Tailwind frontend (reusing CartVerse patterns).
2. Build UC1: single description input → ranked results with confidence badges + reasoning text + persistent disclaimer banner.
3. Build UC4: direct HSN code lookup view.

Day 10

4. Build UC2: CSV upload → progress/spinner → downloadable classified CSV, with flagged-row indication.
5. Build UC3: correction submission UI on a result (e.g., "this isn't right — enter correct code").
6. Wire all four use cases against the live local backend end-to-end. Fix rough edges in loading states and error displays (especially the fallback states from Section 8 — make sure "unranked — explanation unavailable" renders clearly, not as a generic error).

Checkpoint: You can walk through UC1–UC4 in the browser, live, without touching the terminal.

Phase 6: Dockerize (Days 11–12)

Exit criteria: `docker-compose up` runs the full stack.

Day 11

1. Skim Docker's "Get Started" guide (~1 hour) — just enough to apply, not to master.
2. Write Dockerfiles for backend and frontend using the five core instructions: `FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD`.
3. Write `docker-compose.yml` defining backend, frontend, db services. Use service names (not `localhost`) for inter-service communication — the single most common first-timer trap, now avoided in advance.

Day 12

4. Move secrets into `.env` + `env_file`: in compose.
5. Run `docker-compose up` from a clean state (delete local volumes/build cache if needed) and fix whatever breaks — expect at least one iteration on env vars or service networking.
6. Confirm this satisfies the Deployability NFR: zero manual steps beyond `.env` setup.
7. Skip entirely: Kubernetes, multi-stage build optimization, Swarm, non-default networking — none of it is in scope.

Checkpoint: On a fresh clone of the repo, `docker-compose up` (after creating `.env`) brings up a fully working stack with no manual intervention.

Phase 7: Deploy (Day 13)

1. Deploy to Render (recommended in Section 13 for lowest new-learning overhead, matching your CartVerse backend experience) — or Railway if you want the Docker-native deploy practice instead.
2. Confirm the deployed environment matches local: run through UC1–UC4 against the public URL.
3. Point the frontend's env config at the deployed backend URL.
4. Sanity-check cost dashboards (OpenAI/Voyage + Anthropic) against the <\$5 budget.

Checkpoint: A public URL exists and a stranger could use it without your local machine running.

Phase 8: Polish + Demo (Day 14)

1. Build the hand-labeled test set (Section 9): 30–40 descriptions, mixed easy/ambiguous, correct codes looked up manually. Do this now if you haven't been building it incrementally along the way (better: you should have been collecting candidates since Day 4).
2. Run the test set through the classifier, compute:
 - Overall top-1 accuracy
 - Accuracy split into "clear-cut" vs. "genuinely ambiguous" buckets (Risk mitigation — report both, not one blended number)
3. Confirm the $\geq 75\%$ target is met or documented honestly if it isn't, with analysis of where it falls short.
4. Write the README:
 - Architecture diagram (mirror Section 7)
 - AI-scoping decisions: why deterministic retrieval does the heavy lifting and the LLM is scoped narrowly to re-rank/explain — this is the interview-defensible decision
 - Data source URL + download date
 - Explicit out-of-scope list (Section 4) — stated deliberately as scoping discipline, not an apology
 - Accuracy numbers with the clear-cut/ambiguous split
 - Setup instructions (`.env` + `docker-compose up`)
5. Record a short demo video walking through UC1–UC4 plus at least one fallback scenario (e.g., forcing the LLM call to fail and showing the graceful "unranked" degradation) — this demonstrates FR-8 concretely, which is hard to convey in text alone.

Checkpoint: Someone could read only the README and understand what you built and why, without opening the code.

Quick-Reference: Requirement → Phase Map

Requirement	Built in Phase
FR-1, FR-2 (ranked classify + reasoning)	3
FR-3 (bulk CSV)	4
FR-4 (corrections)	4
FR-5 (persisted candidate sets)	3

Requirement	Built in Phase
FR-6 (direct lookup)	3
FR-7 (disclaimer)	3, surfaced in 5
FR-8 (graceful degradation)	3, tested in 5 and 8

Standing Reminders Throughout the Build

- Refer back to Section 4's out-of-scope list any time you're tempted to add fine-tuning, auth, multi-language support, or real-time rate syncing.
- Learn each new tool (embeddings, pgvector, structured prompting, Docker) the day before you need it — not in advance.
- Never let a failed dependency produce a confident-looking wrong answer; silence or a labeled degradation, never a guess dressed as certainty.